

C#.NET @ Sunbeam Infotech

Trainer: Nilesh Ghule



Emp

- Id
- Salary
- CalcIncome()
- DoWork()

EmpUtil

- CalcTotalExp()

Labor $\rightarrow$ Total sal = hours * rate

Manager $\rightarrow$ CalcIncome()

- Salary + Bonus

(Mgrs won't get extra component).

Sales Manager & HR Manager.

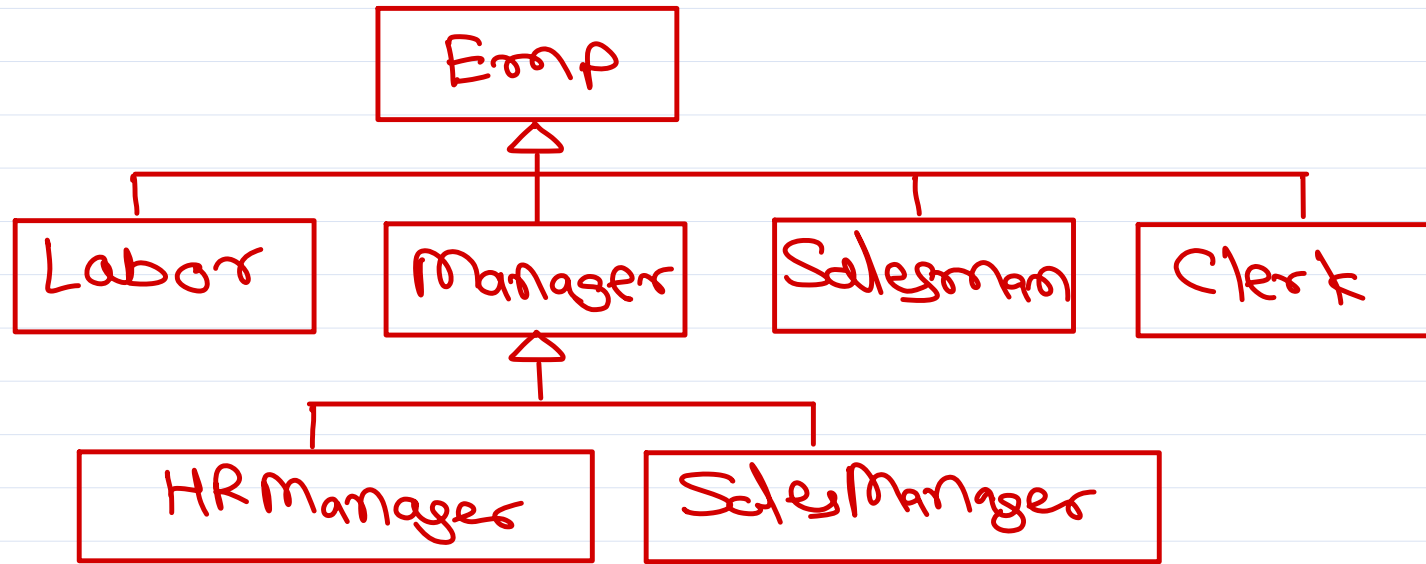
Salesman $\rightarrow$ CalcIncome()

- Salary + Commission

Clerks with no sub-types.



Inheritance



Object Oriented Analysis & Design (OOAD)
- Grady Booch.

* SOLID

* Design Patterns

abstract methods
(contract)

conceptual entity
(no objects)

classes

- fields + methods (reuse).
- virtual methods (polymorphism)
- objects

abstract classes

- fields + methods (reuse).
- virtual methods (polymorphism)



FileEditViewGitProjectBuildDebugTestAnalyzeToolsExtensionsWindowHelp

SearchDay03_Soln

01_Struct01_Struct

GitHub Copilot

Program.csProgram.cs

02_OopInheritance02_OopInheritance.ProgramMain(string[] args)

596061626364656667

0 references

static void Main(string[] args)

{

Emp[] emps = new Emp[5];

// array of 5 references created on heap

// by default each ref is initialized to null

}

}

emps

1000

Array

Length

5

_data

0null

1null

2null

3null

4null

89 %No issues foundLn: 59Ch: 6SPCCRLF

Error ListOutput

Solution Explorer

Search Solution Explorer (Ctrl+;)

Solution 'Day03_Soln' (2 of 2 projects)

01_Struct

02_OopInheritance

Dependencies

Program.cs

Item(s) Saved

0 / 011maindotnet

ENG US08:4125-06-2025

Object Slicing (Java/C#) -

When derived class object is assigned to base class ref, we can only access base class members through that ref.

```
class Person {  
    public string name;  
    // ...  
}
```

```
class Student:Person {  
    public int marks;  
    // ...  
}
```

```
Person p = new Student();  
cw(p.name); // okay  
cw(p.marks); // error
```

Exception - The overridden virtual members are accessed based on type of object (not the ref).

```
Student s = (Student) p;  
cw(s.marks); // okay
```

```
interface MyIntf {  
    void Method();  
}
```

```
class MyClass : MyIntf {  
    public void Method() {  
        // ...  
    }  
}
```

```
class SomeClass {  
    // ...  
}
```

base class



```
class YourClass: SomeClass,  
                MyIntf {  
    // ...  
}
```



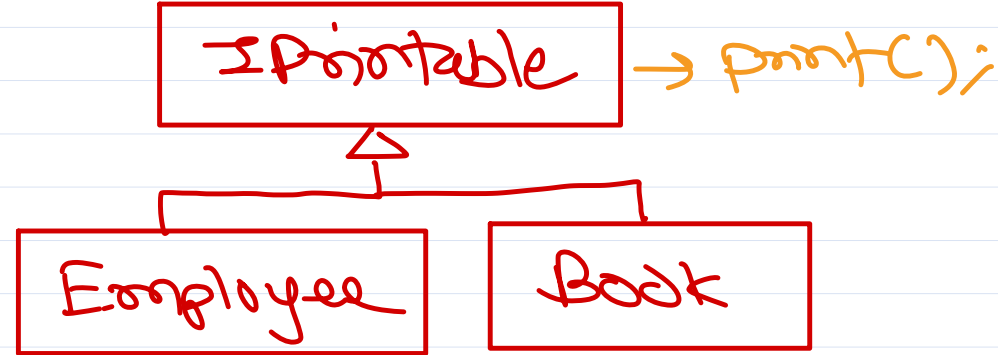
interface(s)

In C# only one base class allowed.
We may implement multiple
interfaces.

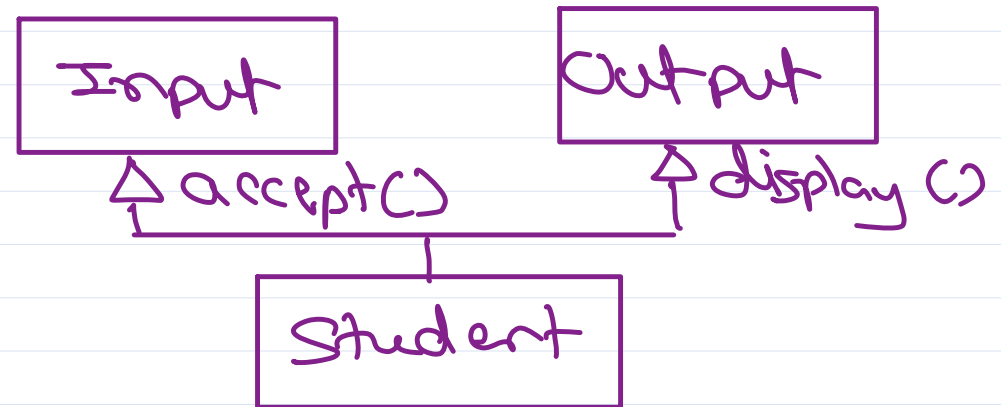
Interfaces

interfaces

- no reusability
(no fields, methods,
no ctors)
- all abstract methods
(contract → standard)
ie. specification → guarantee
that derived class has
the functionality
- can group unrelated classes
(polymorphism)
- avoids fragile base class
problem → immutable
- no inheritance → implementation



one class may impl
multiple interfaces.



Delegates

delegate → like fn alias (in JS) but type safe.
object oriented type safe function pointer.

C → fn pointer →

```
int sum(int a, int b) {  
    return a + b;  
}
```

```
typedef int (*mathop)(int, int);  
mathop ptr = sum;  
res = ptr(22, 7);
```

1000

1000 ptr

declare fn ptr type ①

declare fn ptr & initialize ②

call fn.



Delegates

```
wid sum(int a, int b) {  
    cw(a+b);  
}
```

Step 1 → declare delegate (fn ptr) type.

Syntax: delegate ret-type del-type name (params & types);
delegate wid MathOp(int a, int b);

Step 2 → create delegate obj & init it (with fn addr).

MathOp ptr = new MathOp(Sum); if static
└─> ClassName.Sum
└─> objName.Sum
└─> if non-static
└─> if local fn

Step 3 → call the fn
ptr(22, 7);





Thank you!

Nilesh Ghule <nilesh@sunbeaminfo.com>

